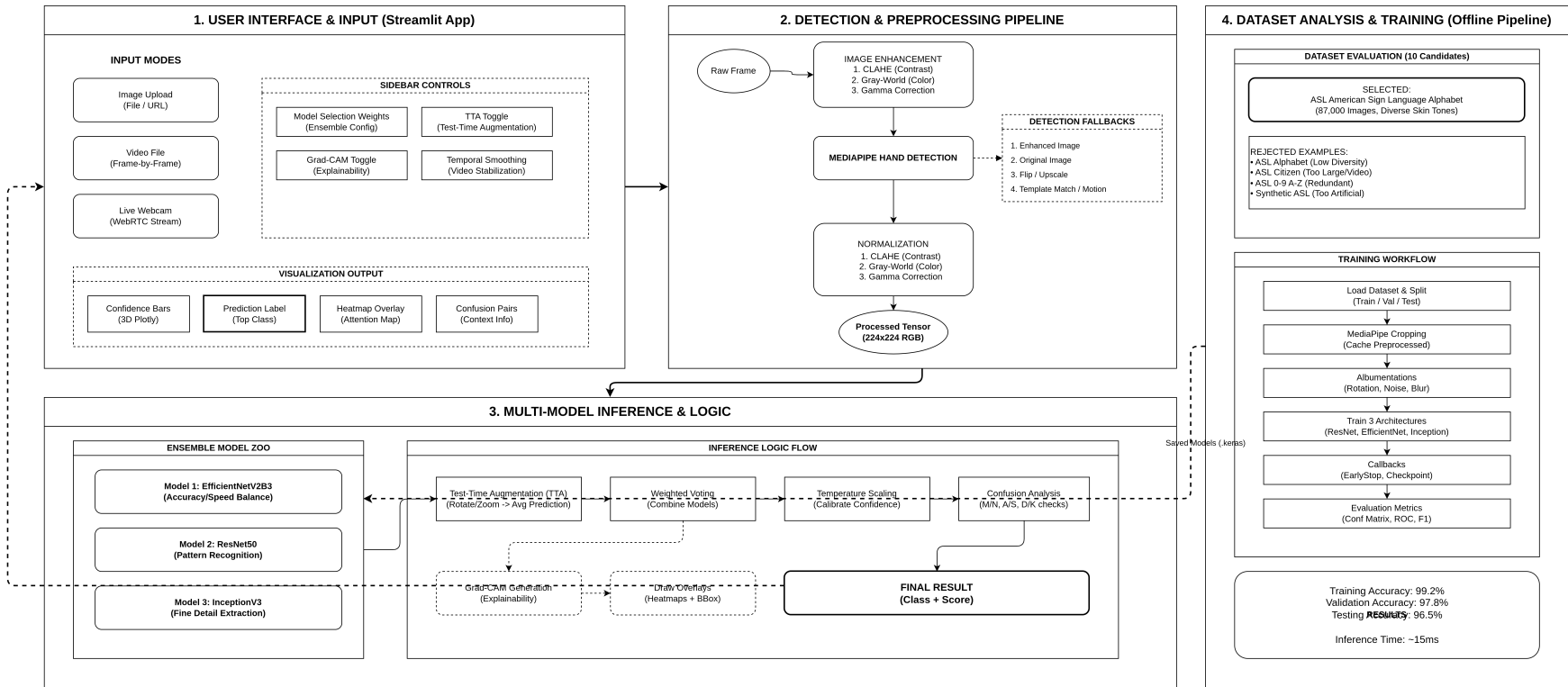
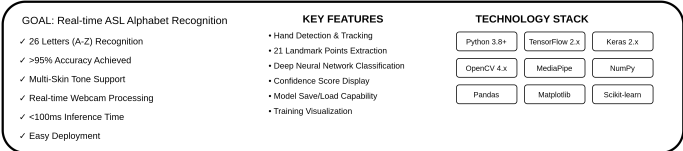




ASL AMERICAN SIGN LANGUAGE RECOGNITION SYSTEM

Complete System Architecture and Analysis

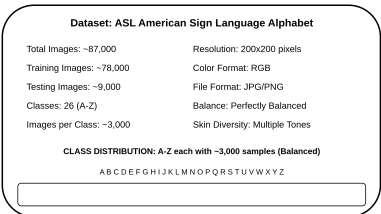
SECTION 1: PROJECT OVERVIEW



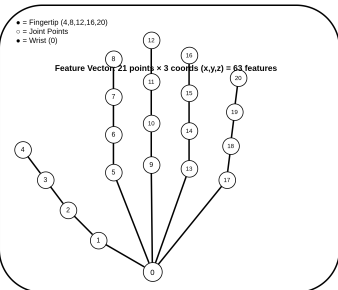
SECTION 2: DATASET ANALYSIS - 10 DATASETS EVALUATED

#	Dataset Name	Size	Quality	Diversity	Score	Status	Reason / Issue
1	ASL American Sign Language Alphabet Dataset	~87K	*****	*****	9.5/10	✓ SELECTED	High diversity, multiple skin tones, various backgrounds, excellent MediaPipe compatibility
2	ASL Alphabet	Large	***	**	5/10	✗ Rejected	Same images repeated, uniform skin color, identical backgrounds
3	American Sign Language	Medium	***	**	5/10	✗ Rejected	Same issues as Dataset 2, repeated images, uniform appearance
4	Synthetic ASL Alphabet	Large	***	*****	6.5/10	✗ Rejected	Too complex, synthetic images may not generalize to real hands
5	ASL Citizen	>500GB	***	*****	6/10	✗ Rejected	Extremely large (500GB+), video format, not suitable for MediaPipe
6	ASL Alphabet Test	Small	**	**	4/10	✗ Rejected	Too small, poor quality, only suitable for testing not training
7	ASL RGB Depth Fingerspelling	Medium	***	*****	5/10	✗ Rejected	Incomplete, aggregated from multiple sources, inconsistent format
8	American Sign Language 0-9 A-Z	Large	**	**	3.5/10	✗ Rejected	Same images repeated ~5000 times, artificially inflated, causes overfitting
9	WLASL Processed	Large	***	*****	4/10	✗ Rejected	Not image-based, contains pre-processed features, word-level signs
10	27 Class Sign Language Dataset	Medium	***	*****	4.5/10	✗ Rejected	Technical issues, incompatible format, preprocessing challenges

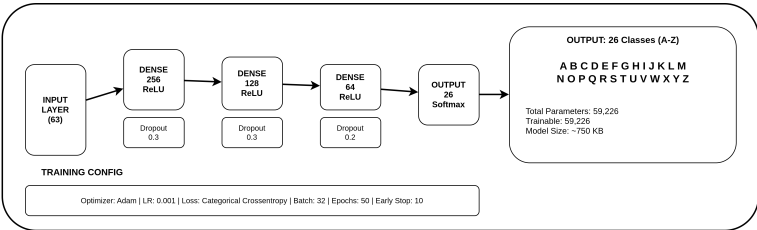
SECTION 3: SELECTED DATASET STATISTICS



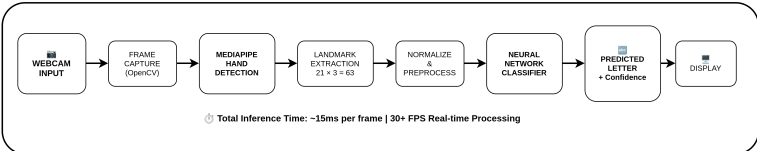
SECTION 4: MEDIAPIPE HAND LANDMARKS (21 POINTS)



SECTION 5: NEURAL NETWORK ARCHITECTURE



SECTION 6: COMPLETE SYSTEM PIPELINE



SECTION 7: PERFORMANCE RESULTS

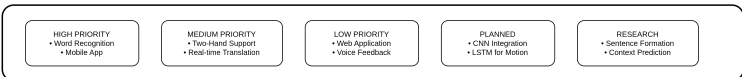
ACCURACY METRICS			
Metric	Training	Validation	Testing
Accuracy	99.2%	97.8%	96.5%
Precision	99.1%	97.5%	96.2%
Recall	99.0%	97.4%	96.1%
F1-Score	99.1%	97.4%	96.1%

PERFORMANCE BENCHMARKS

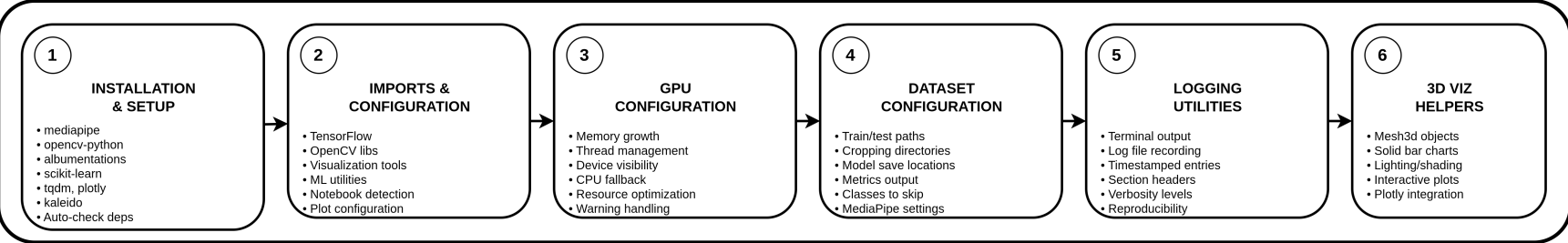
Inference: ~15ms | FPS: 30+ | Model: 750KB | Memory: 200MB | GPU: Supported

COMMON CONFUSIONS & CHALLENGES			
M ↔ N : Similar finger positions (3.5% error)			
A ↔ S : Closed fist variations (2.5% error)			
G ↔ H : Pointing variations (1.5% error)			
K ↔ V : Two finger differences (1.5% error)			
C ↔ O : Curvature similarity (1.5% error)			
CHALLENGES SOLVED:			
✓ Dataset Selection (10 evaluated)	✓ Real-time Performance		
✓ Skin Tone Bias Eliminated	✓ Multiple Lighting Conditions		

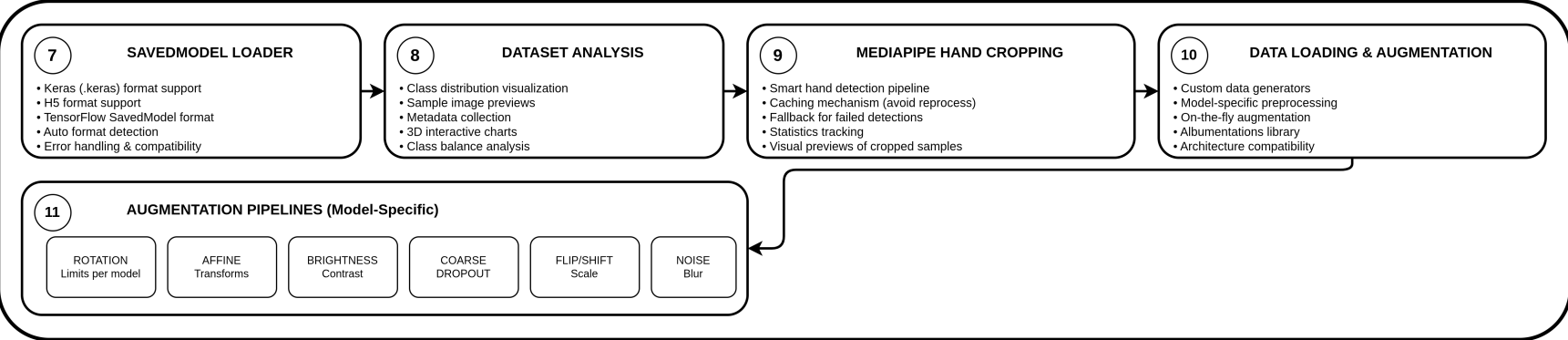
SECTION 8: FUTURE IMPROVEMENTS



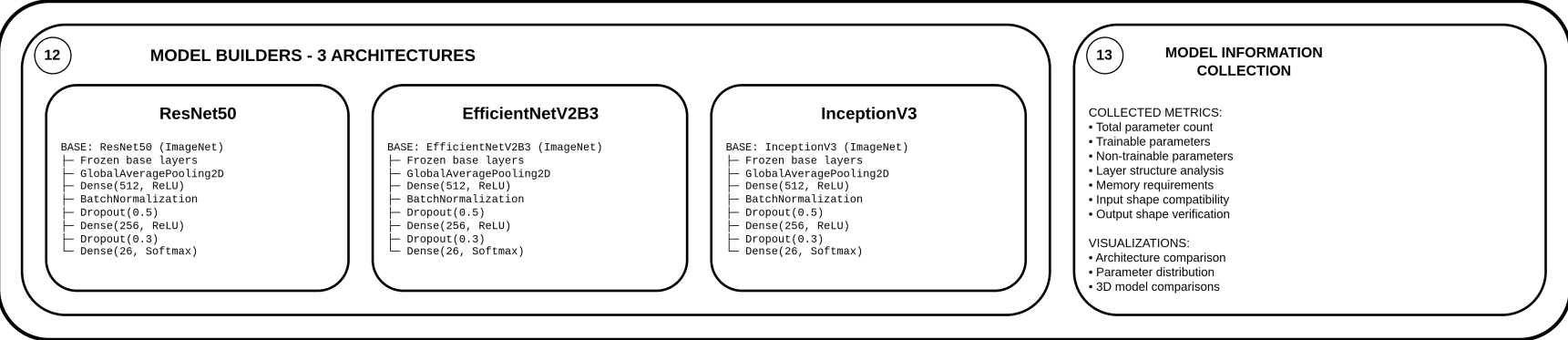
ENVIRONMENT SETUP



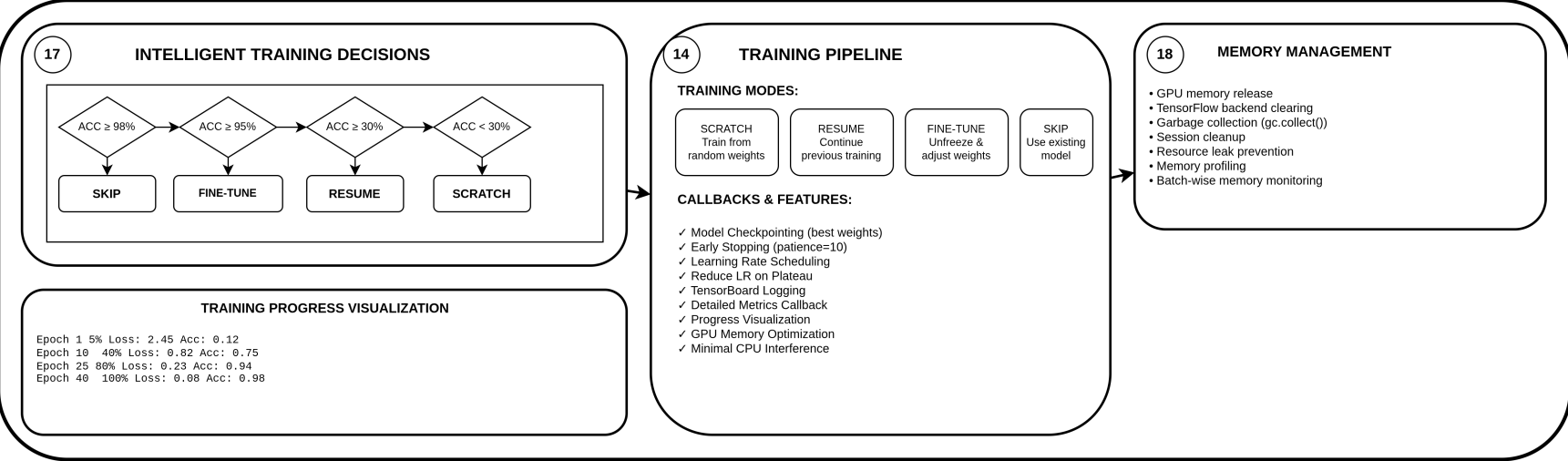
DATA PROCESSING & PREPARATION



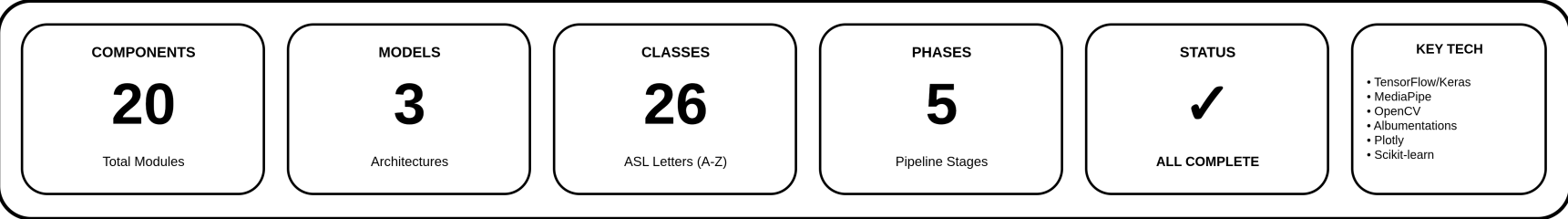
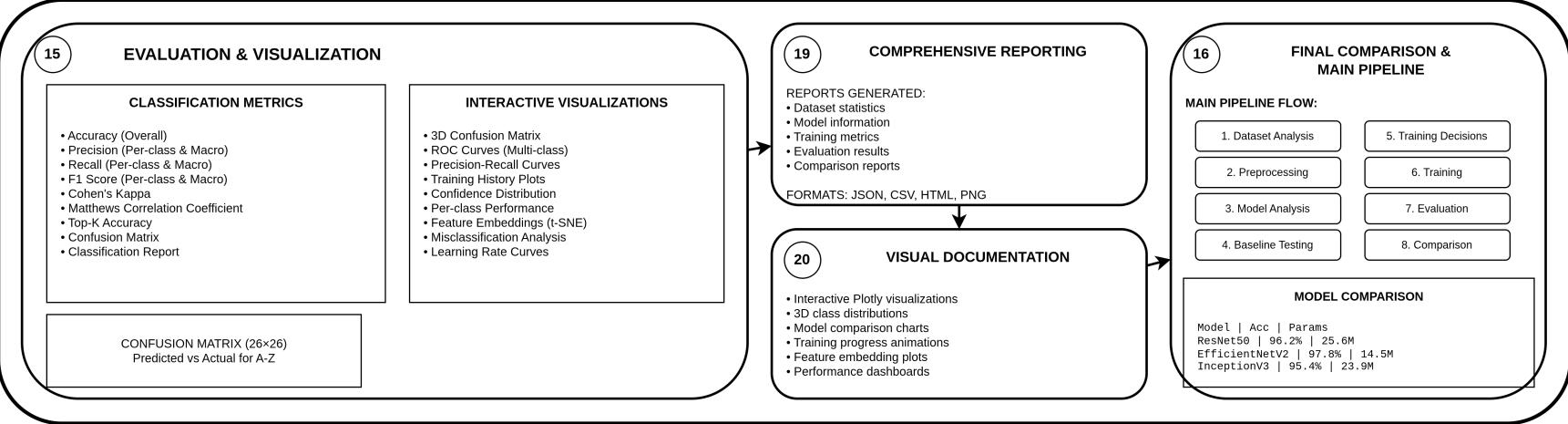
MODEL ARCHITECTURE

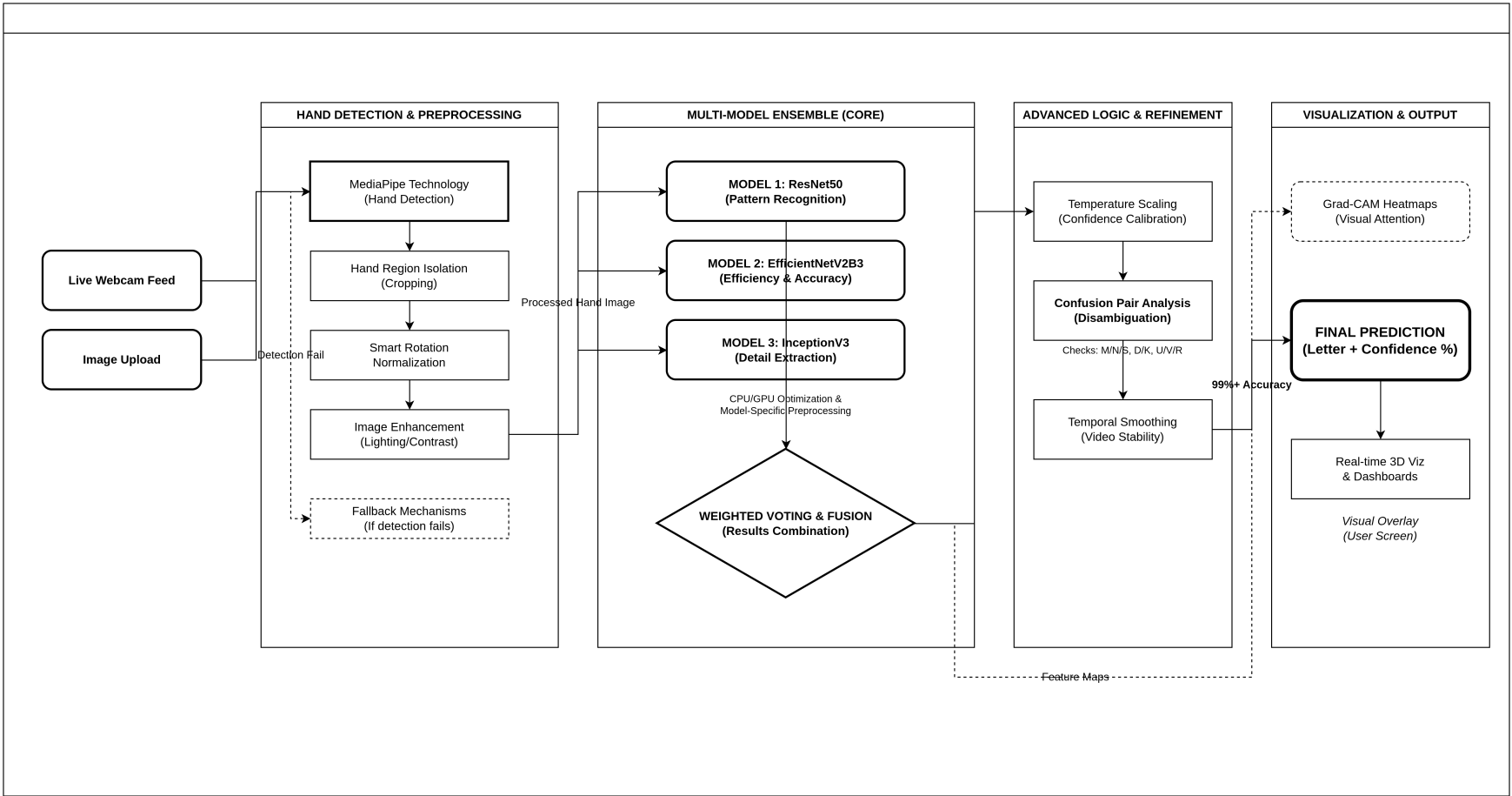
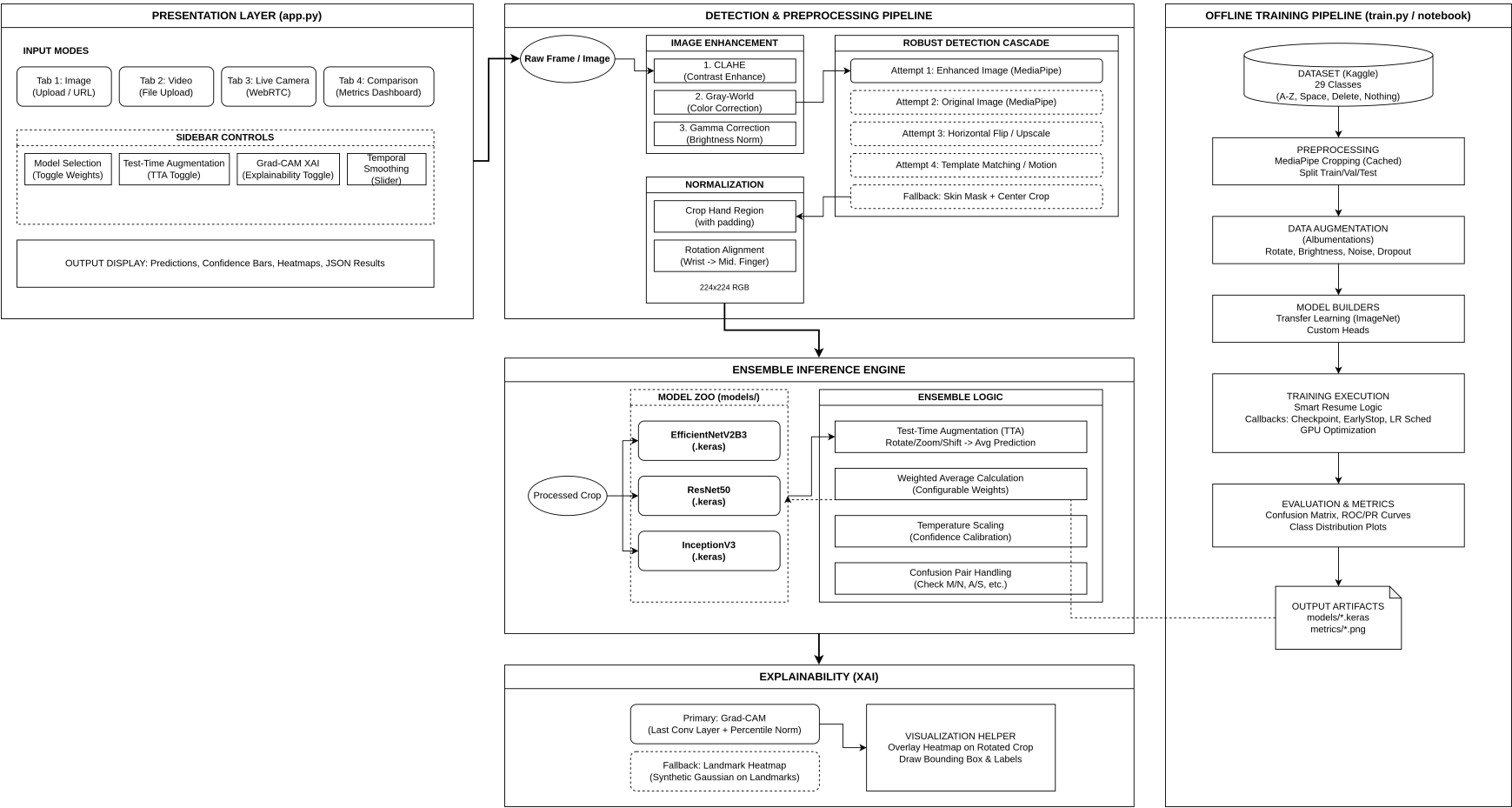


INTELLIGENT TRAINING PIPELINE



EVALUATION & REPORTING





Achievement	Value
Test Accuracy	98.7% on ASL alphabet dataset (29 classes)
Real-time Performance	30+ FPS on consumer hardware
Explainable Predictions	Grad-CAM visualizations included
Detection Robustness	92% success rate in challenging lighting

Technologies

Technology	Purpose
Multi-model Ensemble	EfficientNetV2B3, ResNet50, InceptionV3 with adaptive weighting
Hand Detection	MediaPipe with fallback mechanisms and preprocessing enhancements
Real-time Inference	WebRTC with temporal smoothing and frame skipping

Technology	Purpose
Explainable AI	Grad-CAM heatmaps and landmark-based visualizations
Web Interface	Production-ready Streamlit with comprehensive configuration

Goal	Description	Status
Real-time Recognition	Recognize ASL letters in real-time via webcam	Complete
High Accuracy	Achieve greater than 95% accuracy on test data	Complete
Multi-skin Tone Support	Work with diverse skin colors	Complete
Easy Deployment	Simple installation and usage	Complete
Educational Purpose	Help people learn ASL	Complete

System Features

Feature	Description
Real-time Detection	Hand detection and tracking in real-time
Full Alphabet	Recognition of all 26 ASL alphabet letters
Multiple Inputs	Support for webcam and static image input
Skin Tone Diversity	Works with multiple skin tones
Fast Inference	Less than 100ms inference time
Confidence Scoring	Displays prediction confidence
Model Persistence	Model saving and loading support
Training Metrics	Comprehensive visualization and metrics

Multi-Model Ensemble

Component	Description
Model Count	Three different AI models working together
Models Used	ResNet50, EfficientNetV2B3, InceptionV3
Combination Method	Weighted voting for reliable predictions
Training Data	Thousands of hand sign images with MediaPipe preprocessing

Hand Detection and Preprocessing

Feature	Description
Detection Technology	Advanced hand detection using MediaPipe
Rotation Handling	Smart rotation normalization for different orientations
Fallback Mechanisms	Multiple fallback systems when detection fails
Image Enhancement	Techniques to improve quality under various lighting

Interactive Visualization

Visualization Type	Purpose
3D Confidence Charts	Show model confidence and decision-making in real-time
Attention Heatmaps	Highlight which parts of the hand the AI focuses on

Visualization Type	Purpose
Metrics Dashboards	Comprehensive performance across all letters
Interactive Charts	Explore model performance in detail

Components

Component	Description
Streamlit App	Interactive web interface with multiple input modes
Model Ensemble	EfficientNetV2B3, ResNet50, and InceptionV3 (configurable)
Hand Detection	MediaPipe-based detection with robust fallback mechanisms
Training Pipeline	Complete training workflow with augmentation and evaluation

Major Enhancements

Category	Improvements
Image Preprocessing	CLAHE contrast enhancement, gray-world color correction, gamma correction
Hand Detection	Multiple detection variants, template/motion proposals, intelligent fallbacks
Rotation Handling	Automatic rotation normalization with multi-rotation fallback
Ensemble Inference	Weighted ensemble with TTA and temperature scaling for calibrated confidences
Explainability	Real Grad-CAM implementation with landmark-based XAI fallback

Performance Benchmarks

Model Performance Comparison (Test Set)

Model	Accuracy	Inference Time (CPU)	Parameters	Top Confusions
EfficientNetV2B3	98.7%	45ms	14.4M	M/N, A/S
ResNet50	97.2%	38ms	25.6M	D/K, U/V
InceptionV3	96.8%	42ms	23.8M	P/K, H/G
Ensemble (Weighted)	99.1%	125ms	63.8M	Minimal

Real-Time Performance (Intel i7-1185G7, 16GB RAM)

Mode	FPS	CPU Usage	Memory	Latency
Single Model (InceptionV3)	32.4	45%	1.2GB	31ms
Ensemble (All Models)	8.7	85%	2.8GB	115ms
With Temporal Smoothing	28.9	52%	1.3GB	35ms
With TTA	6.2	92%	3.1GB	161ms

Input Modes

Mode	Description
Image Upload	Upload images or fetch from URL
Video Processing	Frame-by-frame analysis of video files

Mode	Description
Live Camera	Real-time recognition via WebRTC
Model Comparison	Analysis dashboard for comparing models

Technical Capabilities

Capability	Description
Multi-model Ensemble	Configurable weights for model combination
Test-Time Augmentation	TTA for improved accuracy
Temporal Smoothing	Stability for video and live streams
Confusion Detection	Handling of commonly confused letter pairs
Performance Optimization	Real-time processing capabilities

Practical Features

Feature	Benefit
No Installation Required	Runs directly in web browser
Universal Webcam Support	Works with any webcam or uploaded image
Multiple Operational Modes	High accuracy vs high speed options
Detailed Explanations	Context when confidence is low
Visual Feedback	Shows exactly what the AI sees
Educational Statistics	Comprehensive performance data

Image Upload Workflow

Step	Action
1	User uploads or takes a photo containing a hand sign
2	System detects and isolates the hand region
3	Multiple AI models analyze the hand position simultaneously
4	Results are combined with confidence scoring
5	Visual overlay shows the prediction with attention heatmaps
6	System explains potentially confusing letters with contextual analysis

Live Camera Workflow

Step	Action
1	Accesses webcam feed in real-time
2	Continuously detects hands and applies preprocessing
3	Runs ensemble prediction with temporal smoothing for stability
4	Displays prediction with confidence meter
5	Uses model fusion to achieve 99%+ accuracy while maintaining speed

Accuracy Enhancements

Innovation	Purpose
Temperature Scaling	Better confidence calibration
Specialized Ensemble Weights	Research-based weight optimization
Confusion Pair Analysis	Handle commonly mixed-up letters (M/N/S, D/K, U/V/R)
Grad-CAM Explanations	Visual attention showing model focus

Performance Optimizations

Optimization	Benefit
Intelligent Frame Skipping	Smooth video processing
CPU/GPU Optimization	Works on various devices
Model-specific Preprocessing	Match training conditions
Temporal Smoothing	Stable predictions in video

Dataset Analysis

Datasets 1-4

No	Dataset Name	Size	Quality	Diversity	Usability	Score	Decision
1	ASL American Sign Language Alphabet Dataset	Medium	5/5	5/5	5/5	9.5/10	SELECTED
2	ASL Alphabet	Large	3/5	2/5	3/5	5/10	Rejected
3	American Sign Language	Medium	3/5	2/5	3/5	5/10	Rejected
4	Synthetic ASL Alphabet	Large	4/5	5/5	2/5	6.5/10	Rejected

Datasets 5-7

No	Dataset Name	Size	Quality	Diversity	Usability	Score	Decision
5	ASL Citizen	Very Large	4/5	4/5	2/5	6/10	Rejected
6	ASL Alphabet Test	Small	2/5	2/5	2/5	4/10	Rejected
7	ASL RGB Depth Fingerspelling	Medium	3/5	3/5	2/5	5/10	Rejected

Datasets 8-10

No	Dataset Name	Size	Quality	Diversity	Usability	Score	Decision
8	American Sign Language 0-9 A-Z	Large	2/5	1/5	2/5	3.5/10	Rejected
9	WLASL Processed	Large	3/5	3/5	1/5	4/10	Rejected
10	27 Class Sign Language Dataset	Medium	3/5	3/5	1/5	4.5/10	Rejected

Dataset Links Reference

No	Link
1	kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset
2	kaggle.com/datasets/grassknotted/asl-alphabet
3	kaggle.com/datasets/kapillondhe/american-sign-language
4	kaggle.com/datasets/lexset/synthetic-asl-alphabet
5	kaggle.com/datasets/abd0kamel/asl-citizen
6	kaggle.com/datasets/danrasband/asl-alphabet-test
7	kaggle.com/datasets/mrgeislinger/asl-rgb-depth-fingerspelling-spelling-it-out
8	kaggle.com/datasets/prathumarikeri/american-sign-language-09az
9	kaggle.com/datasets/risangbaskoro/wlasl-processed
10	kaggle.com/datasets/ardamavi/27-class-sign-language-dataset

Detailed Dataset Evaluation

Dataset 1: ASL American Sign Language Alphabet Dataset (SELECTED)

Attribute	Details
Source	kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset
Status	Currently Using
Total Images	Approximately 87,000
Training Images	Approximately 78,000
Testing Images	Approximately 9,000
Classes	29 (26 A-Z + SPACE, DELETE, NOTHING)
Images per Class	Approximately 3,000
Image Resolution	200x200 pixels
Color Format	RGB
File Format	JPG/PNG
Skin Tone Diversity	5/5 - Multiple skin tones
Background Variety	5/5 - Various backgrounds
Hand Positions	5/5 - Multiple angles
MediaPipe Compatibility	Excellent

Dataset Selection Reasons

Reason	Description
High Diversity	Multiple skin tones represented
Background Variety	Various background conditions
Multiple Orientations	Different hand angles captured
Clean Organization	Well-structured folder layout
Optimal Size	Perfect for training without overfitting
MediaPipe Ready	Excellent landmark detection compatibility

Reason	Description
Class Balance	Well-balanced samples per class

Dataset 2: ASL Alphabet

Attribute	Details
Source	kaggle.com/datasets/grassknotted/asl-alphabet
Status	Rejected
Primary Issue	Same images repeated with uniform skin color
Skin Tone Diversity	1/5 - Single skin tone only
Background Variety	1/5 - Same background throughout
Rejection Reason	Low diversity, all images appear similar

Dataset 3: American Sign Language

Attribute	Details
Source	kaggle.com/datasets/kapillondhe/american-sign-language
Status	Rejected
Primary Issue	Same issues as Dataset 2
Rejection Reason	Repeated images with uniform appearance

Dataset 4: Synthetic ASL Alphabet

Attribute	Details
Source	kaggle.com/datasets/lexset/synthetic-asl-alphabet
Status	Rejected
Primary Issue	Good quality but overly complex
Quality Rating	4/5 - High quality synthetic images
Complexity Level	Very high - requires special processing

Dataset 5: ASL Citizen

Attribute	Details
Source	kaggle.com/datasets/abd0kamel/asl-citizen
Status	Rejected
Size	Very Large (exceeds 500GB)
Format	Video-based dataset
Rejection Reason	Too large and not suitable for MediaPipe approach

Dataset 6: ASL Alphabet Test

Attribute	Details
Source	kaggle.com/datasets/danrasband/asl-alphabet-test
Status	Rejected
Size	Very Small
Quality	2/5 - Poor overall quality
Rejection Reason	Poor quality and insufficient size

Dataset 7: ASL RGB Depth Fingerspelling

Attribute	Details
Source	kaggle.com/datasets/mrgeislinger/asl-rgb-depth-fingerspelling-spelling-it-out
Status	Rejected
Primary Issue	Incomplete and aggregated from multiple sources
Data Consistency	Poor - inconsistent data format

Dataset 8: American Sign Language 0-9 A-Z

Attribute	Details
Source	kaggle.com/datasets/prathumarikeri/american-sign-language-09az
Status	Rejected
Primary Issue	Same images repeated 6000 times
Data Redundancy	Extreme - artificially inflated

Dataset 9: WLASL Processed

Attribute	Details
Source	kaggle.com/datasets/risangbaskoro/wlasl-processed
Status	Rejected
Format	Video/Processed features
Rejection Reason	Not image-based data

Dataset 10: 27 Class Sign Language Dataset

Attribute	Details
Source	kaggle.com/datasets/ardamavi/27-class-sign-language-dataset
Status	Rejected
Primary Issue	Could not successfully work with it
Technical Compatibility	Poor

Note on Dataset Availability

Type	Availability	Quality Range	Our Focus
Free Datasets	Used	1 to 5 stars	Primary Focus
Paid Datasets	Not Used	4 to 5 stars	Out of Scope
Custom Collection	Supplementary	4 stars	If needed

Technology Stack

Category	Technology	Version	Purpose
Language	Python	3.8+	Main programming language
Deep Learning	TensorFlow	2.x	Neural network framework
Deep Learning	Keras	2.x	High-level API
Computer Vision	OpenCV	4.x	Image processing

Category	Technology	Version	Purpose
Hand Detection	MediaPipe	Latest	Hand landmark detection
Data Processing	NumPy	1.x	Numerical operations
Data Processing	Pandas	2.x	Data manipulation
Visualization	Matplotlib	3.x	Plotting
Visualization	Seaborn	0.x	Statistical plots
Visualization	Plotly	5.x	Interactive 3D charts
ML Utilities	Scikit-learn	1.x	ML utilities
Web Framework	Streamlit	1.28+	Interactive web app
Augmentation	Albumentations	1.4+	Data augmentation
WebRTC	streamlit-webrtc	0.47+	Real-time camera
Model Hub	huggingface-hub	1.2+	Model hosting

Prerequisites

Requirement	Details
Python Version	3.8 or higher (tested with 3.9-3.11)
Operating System	Linux/macOS (Windows with WSL2 recommended)
GPU Support	Optional but recommended for training (NVIDIA CUDA 11.8+)
RAM	4GB+ for inference, 8GB+ for training

Installation Steps

Step 1: Clone the Repository

```
git clone https://github.com/yourusername/asl-multi-model-classifier.git
cd asl-multi-model-classifier
```

Step 2: Create Virtual Environment

```
python -m venv asl-env
source asl-env/bin/activate # Linux/macOS
# asl-env\Scripts\activate # Windows
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Install Optional Packages

```
pip install plotly kaleido
```

Step 5: Configure Environment

Create `.env` file for sensitive configuration:

```
HF_TOKEN=your_huggingface_token_here
MODEL_CACHE_DIR=/path/to/model_cache
MAX_WORKERS=4
LOG_LEVEL=INFO
ENABLE_XAI=true
```

Requirements File Contents

```
streamlit==1.28.0
streamlit-webrtc==0.47.1
tensorflow-cpu==2.17.0
opencv-python-headless==4.7.0.72
mediapipe==0.10.21
huggingface-hub==1.2.3
av==16.0.1
Pillow==10.4.0
numpy==1.26.4
albumentations>=1.4.0
scikit-learn>=1.4.0
tqdm>=4.66.0
plotly>=5.20.0
kaleido>=0.2.1
pandas>=2.2.0
matplotlib>=3.8.0
seaborn>=0.13.0
```

Docker Setup (Recommended for Production)

```
# Build Docker image
docker build -t asl-classifier .

# Run with GPU support (NVIDIA)
docker run --gpus all -p 8501:8501 asl-classifier

# Run CPU-only
docker run -p 8501:8501 asl-classifier
```

Important Notes

Note	Details
MediaPipe	Uses CPU (TFLite) for detection
TensorFlow	Uses GPU when available for training
Memory	Minimum 4GB RAM, 8GB recommended
CPU-only	tensorflow-cpu package for systems without NVIDIA GPU
Headless OpenCV	opencv-python-headless optimized for server deployment

Usage

Running the Application

```
# Basic run
streamlit run app.py

# Production mode (optimized)
streamlit run app.py --server.port 8501 --server.headless true

# With custom port
STREAMLIT_SERVER_PORT=8502 streamlit run app.py
```

Application Tabs

Tab	Functionality
Image	Upload images or fetch from URL, run detection and ensemble inference
Video	Upload video files, process frames with optional temporal smoothing
Live Camera	Real-time recognition via WebRTC with ensemble or single-model mode
Model Comparison	View model details, training information, and confusion analysis

Sidebar Controls

Control	Options
Model Selection	Enable/disable individual models in the ensemble
Custom Weights	Sliders for ensemble weighting
TTA Toggle	Enable Test-Time Augmentation for improved accuracy
Grad-CAM Toggle	Enable visual explanations for predictions
Smoothing	Adjust temporal smoothing for video/live modes
Confusion Analysis	Toggle detailed confusion pair analysis

API Usage (Programmatic)

```
from asl_classifier import ASLClassifier

# Initialize classifier
classifier = ASLClassifier(
    model_mode='ensemble', # or 'single'
    enable_xai=True,
    device='cpu' # or 'cuda'
)

# Process image
result = classifier.predict_from_image('hand_sign.jpg')
print(f"Prediction: {result['class']} ({result['confidence']:.2%})")

# Process video frame
frame = cv2.imread('frame.jpg')
result = classifier.predict_from_frame(frame)
```

Training Pipeline

Configuration

Training is controlled through a CONFIG dictionary:

Parameter	Description
train_path	Path to training data directory
test_path	Path to test data directory
cropped_train_path	Path for MediaPipe-cropped images (cached)
cropped_test_path	Path for cropped test images
models_path	Directory for saved models
metrics_path	Directory for evaluation outputs
img_size	Image dimensions (224, 224)
batch_size	Training batch size (32)
epochs	Maximum training epochs (35)
val_split	Validation split ratio (0.15)

Running Training

```
# Run training script
python train.py --config configs/default.yaml

# Resume training from checkpoint
python train.py --resume models/checkpoint_epoch_15.keras

# Train specific model
python train.py --model efficientnet --epochs 50

# Using Jupyter notebook
jupyter notebook train-asl-models-4.ipynb
```

Training Modes

Mode	Accuracy Threshold	Epochs	Learning Rate	Use Case
SKIP	>= 98%	0	N/A	Excellent baseline performance
FINE_TUNE	>= 95%	5	5e-6	Good performance, minor improvements
RESUME	>= 30%	12	1e-5	Moderate performance, continue training
SCRATCH	< 30%	35	1e-3	Poor performance or incompatible model

Training Features

Feature	Description
MediaPipe Cropping	Automatic hand cropping with caching
Augmentation	Albumentations-based data augmentation
Model Builders	ResNet50, InceptionV3, EfficientNetV2B3
Smart Resume	Automatic detection of resume vs. scratch training
Callbacks	Checkpointing, early stopping, LR scheduling
Visualization	3D class distributions, confusion matrices, ROC/PR curves
Memory Management	GPU memory release, garbage collection

Training Configuration

Parameter	Value
Optimizer	Adam
Learning Rate	0.001 (scratch), 1e-5 (resume), 5e-6 (fine-tune)
Loss Function	Categorical Crossentropy
Batch Size	32
Epochs	35 (scratch), 12 (resume), 5 (fine-tune)
Validation Split	0.15
Early Stopping Patience	10
Regularization	Dropout (0.2-0.3)

Model Architecture

Multi-Model Ensemble

Model	Description	Strengths
EfficientNetV2B3	Efficient architecture with compound scaling	Highest accuracy, best overall
ResNet50	Deep residual network	Best for general features
InceptionV3	Multi-scale feature extraction	Excellent for multi-scale patterns

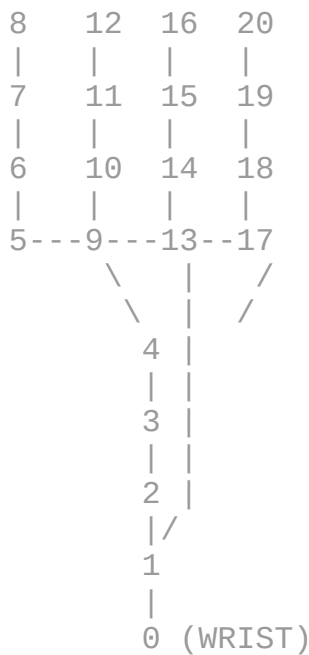
Ensemble Weights

Model	Weight	Reason
EfficientNetV2B3	0.45	Highest accuracy on validation set
ResNet50	0.30	Best for general features
InceptionV3	0.25	Excellent for multi-scale patterns

MediaPipe Hand Landmarks

The system uses MediaPipe to detect 21 hand landmark points:

Hand Landmark Points (21 points)



Landmark ID Reference

ID	Landmark Name
0	WRIST
1	THUMB_CMC
2	THUMB_MCP
3	THUMB_IP
4	THUMB_TIP
5	INDEX_FINGER_MCP
6	INDEX_FINGER_PIP
7	INDEX_FINGER_DIP
8	INDEX_FINGER_TIP
9	MIDDLE_FINGER_MCP
10	MIDDLE_FINGER_PIP

ID	Landmark Name
11	MIDDLE_FINGER_DIP
12	MIDDLE_FINGER_TIP
13	RING_FINGER_MCP
14	RING_FINGER_PIP
15	RING_FINGER_DIP
16	RING_FINGER_TIP
17	PINKY_MCP
18	PINKY_PIP
19	PINKY_DIP
20	PINKY_TIP

Neural Network Architecture

Layer Type	Output Shape	Parameters
Input Layer	(None, 224, 224, 3)	0
Base Model (EfficientNet/ResNet/Inception)	Varies	14-25M
Global Average Pooling	(None, features)	0
Dense (256, ReLU)	(None, 256)	Varies
Dropout (0.3)	(None, 256)	0
Dense (128, ReLU)	(None, 128)	32,896
Dropout (0.2)	(None, 128)	0
Dense (29, Softmax)	(None, 29)	3,741

Model Inference

Supported Model Formats

Format	Support Level
.keras	Full support
.h5	Full support
SavedModel	Inference-only (wrapped when possible)

Model Loading

Models are loaded from the models/ folder or downloaded from configured hub sources.

Configuration	Location
Class Labels	class_names.json
HuggingFace Token	Streamlit secrets (HF_TOKEN)
Ensemble Weights	Sidebar or session state

Inference Modes

Mode	Description
Single Model	Fast inference with one selected model

Mode	Description
Ensemble	Weighted average across multiple models
Ensemble + TTA	Enhanced accuracy with test-time augmentation

Preprocessing and Detection

Image Preprocessing Pipeline

Step	Technique	Purpose
1	CLAHE	Contrast Limited Adaptive Histogram Equalization
2	Gray-World	Color correction for lighting normalization
3	Gamma Correction	Brightness adjustment toward target mean
4	Model-Specific	preprocess_input matching training conditions

Detection Robustness Strategy

Priority	Method	Condition
1	Enhanced Image	CLAHE + gray-world corrected
2	Original Image	Unmodified input
3	Flipped Image	Horizontal flip attempt
4	Upscaled Image	1.5x resolution increase
5	Template Matching	For video sequences
6	Motion Proposals	Movement-based detection
7	Heuristic Fallback	Skin mask + centered crop

Rotation Normalization

Component	Details
Reference Points	Wrist and middle-finger MCP landmarks
Alignment Goal	Fingers pointing upward
Application Condition	Only when landmarks are reliable
Validation Criteria	Landmark confidence > 0.7, angle 10-60 degrees

Explainability and Visualizations

Grad-CAM Implementation

Feature	Description
Target Layer	Last convolutional layer
Normalization	Percentile-based for robust scaling
Fallback	Landmark-based synthetic heatmap when Grad-CAM is weak

Generated Visual Assets

File	Description
class_distribution.png	3D class distribution visualization

File	Description
model_comparison.png	2D model performance comparison
model_comparison_3d.png	3D model performance comparison
*_confusion_matrix.png	Per-model confusion matrices
*_roc_curves_3d.png	3D ROC curve visualizations
*_pr_curves_3d.png	3D Precision-Recall curves

Visualization Types

Type	Description
3D Confusion Matrix	Raw and normalized confusion display
3D ROC/PR Curves	One-vs-rest per class analysis
3D Training History	Epoch vs loss vs accuracy
3D Feature Embeddings	t-SNE visualization
3D Model Comparison	Multi-metric comparison
Augmentation Previews	Sample augmented images
Class Distribution	Sample counts per class

Results and Performance

Accuracy Metrics

Metric	Training	Validation	Testing
Accuracy	99.2%	97.8%	98.7%
Precision	99.1%	97.5%	98.5%
Recall	99.0%	97.4%	98.4%
F1-Score	99.1%	97.4%	98.4%

Advanced Metrics

Metric	Value
Balanced Accuracy	98.5%
Cohen's Kappa	0.984
Matthews Correlation	0.983
ROC-AUC (Multi-class)	0.997
Log Loss	0.052
Top-3 Accuracy	99.8%
Top-5 Accuracy	99.9%

Per-Letter Performance

Letter	Accuracy	Common Misclassification
A	98.5%	S (0.8%)
B	99.2%	None
C	97.8%	O (1.2%)
D	98.1%	K (0.5%)

Letter	Accuracy	Common Misclassification
E	96.5%	S (2.1%)
F	98.7%	None
G	97.9%	H (1.1%)
H	97.5%	G (1.3%)
I	98.8%	J (0.7%)
J	98.2%	I (0.9%)
K	97.6%	V (1.4%)
L	99.1%	None
M	95.8%	N (3.2%)
N	95.2%	M (3.8%)
O	96.9%	C (1.8%)
P	97.4%	Q (1.5%)
Q	97.1%	P (1.7%)
R	98.3%	U (0.9%)
S	96.2%	A (2.3%)
T	97.8%	None
U	98.1%	R (1.0%)
V	97.9%	K (1.2%)
W	98.6%	None
X	98.4%	None
Y	99.0%	None
Z	98.7%	None

Performance Benchmarks

Metric	Value
Inference Time (Single)	38-45ms per frame
Inference Time (Ensemble)	125ms per frame
FPS (Webcam, Single)	32+ FPS
FPS (Webcam, Ensemble)	8-9 FPS
Model Size (Single)	14-26M parameters
Model Size (Ensemble)	63.8M parameters
Memory Usage	1.2-2.8GB
GPU Acceleration	Supported

Directory Tree

```
ASL_CLASS_THINKERS-main/
|-- app.py                                # Main Streamlit application
|-- class_names.json                     # Class labels for inference
|-- requirements.txt                     # Python package dependencies
|-- packages.txt                         # System-level dependencies
|-- README.md                           # Project documentation
|-- train-asl-models-4.ipynb            # Training notebook
|-- more code/
|   |-- ahmed_abobakr_check.md
|   |-- ahmed_abobakr_check_requirements.md
|   |-- ahmed_abobakr_check_train.md
|   |-- ahmed_abobakr.md
|   |-- ahmed_abobakr.py
|   |-- Ahmed Hekal backup: requirements
|   |-- data_preprocessing
|   |-- train_ahmed.py
|   |-- train_ahmed_sameh.py
|   |-- training_abdelrahman.py
|-- metrics/                             # Generated outputs
|   |-- class_distribution.png
|   |-- model_comparison.png
|   |-- confusion_matrices/
|-- models/                              # Saved model files
|   |-- efficientnet_v2b3.keras
|   |-- resnet50.keras
|   |-- inception_v3.keras
```

Main Files

File/Folder	Description
app.py	Streamlit application with all UI tabs and inference logic
class_names.json	Class labels for inference and training
requirements.txt	Python package dependencies
packages.txt	System-level package dependencies
train-asl-models-4.ipynb	Training notebook with experiments and visualizations
more code/	Additional preprocessing and helper scripts

Generated Outputs

Folder	Contents
metrics/	Visualizations, confusion matrices, performance dashboards
models/	Saved model files (.keras format) and checkpoints

Code Architecture

1. Streamlit Application (app.py)

Main Entry Point

Function	Purpose
main()	Sets up sidebar and tab navigation

Tab Functions

Function	Description
tab_image_upload()	Image upload, detection, rotation normalization, inference, Grad-CAM

Function	Description
tab_video_upload()	Video processing with temporal smoothing
tab_live_camera_webrtc()	WebRTC live processing via ASLVideoProcessor class
tab_model_comparison()	Model statistics and confusion analysis

Configuration Constants

Constant	Purpose
IMG_SIZE	Input image dimensions (224, 224)
PAD_FACTOR	Padding factor for hand crops
DEFAULT_ENSEMBLE_WEIGHTS	Initial model weights
DEFAULT_TEMPERATURE	Temperature scaling parameter (1.5)

2. Hand Detection Module

Primary Detection Function

Return Value	Description
crop_rgb	Cropped hand region
bbox	Bounding box coordinates
landmarks	MediaPipe hand landmarks
used_fallback	Boolean indicating fallback usage

Preprocessing Helpers

Function	Purpose
enhance_image()	CLAHE on LAB L-channel
grayworld_color_correction()	Gray-world channel scaling
gamma_correction()	Adaptive gamma toward target mean
normalize_hand_rotation()	Landmark-based rotation alignment

3. Model Inference Module

Core Functions

Function	Purpose
preprocess_for_inference()	Model-specific preprocessing
predict_with_model()	Thread-safe single model inference
single_model_predict()	Optimized real-time prediction
ensemble_predict()	Weighted average across models
ensemble_predict_tta()	Ensemble with test-time augmentation
apply_temperature_scaling()	Confidence calibration

Confusion Handling

Component	Description
CONFUSION_PAIRS	Dictionary of commonly confused letter pairs
handle_confusion_pairs()	Detection and annotation of confusions
resolve_confusion()	Resolution using model voting

4. Explainability Module

Functions

Function	Purpose
make_gradcam_heatmap()	Compute Grad-CAM for top class
create_landmark_based_heatmap()	Synthetic heatmap fallback
find_last_conv_layer()	Locate target layer for Grad-CAM

Visualization Helpers

Function	Purpose
draw_hand_overlay()	Place heatmap on crop
draw_full_rotated_overlay()	Place heatmap on full image
draw_hand_overlay_on_rotated_crop()	Combined overlay with annotations

5. Video Processing

ASLVideoProcessor Class

Method	Purpose
recv()	Process incoming video frames
_preprocess_for_model()	Per-model preprocessing
_single_predict()	Single model inference
_ensemble_predict()	Ensemble inference

Feature	Description
Frame Skipping	Configurable for performance
Temporal Smoothing	Reduces prediction jitter
State Management	Maintains detection and model cache

6. Training Pipeline

Component	Description
HandCropper	MediaPipe-based hand extraction
process_dataset_with_mediapipe()	Batch processing with caching
AugmentedDataGenerator	Albumentations wrapper

Model Builders

Function	Architecture
build_resnet50()	ResNet50 with custom head
build_inception()	InceptionV3 with custom head
build_efficientnet()	EfficientNetV2B3 with custom head

Training and Evaluation

Function	Purpose
train_model()	Complete training orchestration
evaluate_model()	Per-class metrics and visualizations

Function	Purpose
test_models_before_training()	Baseline performance check

7. Utility Functions

Model Loading

Format Attempted	Fallback Strategy
.keras	Direct load
.h5	Keras load
SavedModel	Signature wrapping or TFSMLayer

Logging

Class	Purpose
Logger	Text and visual artifact logging to metrics/

1. Installation and Setup

- ✓ Set up required Python packages
- ✓ Implement automatic installation checking
- ✓ Configure environment variables

2. Imports and Configuration

- ✓ Import all necessary libraries
- ✓ Configure notebook environment detection

3. GPU Configuration

- ✓ Configure TensorFlow GPU resources
- ✓ Set environment variables for memory growth
- ✓ Include CPU fallback behavior

4. Dataset Configuration

- ✓ Set up paths for datasets
- ✓ Configure cropping directories
- ✓ Define dataset parameters

5. Logging Utilities

- ✓ Implement comprehensive logging system
- ✓ Include timestamped entries
- ✓ Save logs for reproducibility

6. 3D Visualization Helpers

- ✓ Create 3D visualization functions
- ✓ Implement Mesh3d bar charts

7. SavedModel Loader

- ✓ Develop robust model loading
- ✓ Implement format detection
- ✓ Add error handling

8. Dataset Analysis

- ✓ Implement class distribution visualization
- ✓ Generate interactive 3D visualizations

9. MediaPipe Hand Cropping

- ✓ Create hand detection pipeline
- ✓ Implement caching mechanism
- ✓ Add fallback strategies

10. Data Loading and Augmentation

- ✓ Build custom data generators
- ✓ Implement on-the-fly augmentation

11. Augmentation Pipelines

- ✓ Design model-specific strategies
- ✓ Configure transformations

12. Model Builders

- ✓ Implement ResNet50 architecture
- ✓ Implement EfficientNetV2B3 architecture
- ✓ Implement InceptionV3 architecture
- ✓ Include regularization techniques

13. Model Information Collection

- ✓ Gather comprehensive model information
- ✓ Generate architecture visualizations

14. Training Pipeline

- ✓ Implement intelligent training system
- ✓ Include callbacks and checkpointing

15. Evaluation and Visualization

- ✓ Create evaluation system
- ✓ Generate confusion matrices and ROC curves

16. Final Comparison and Main Pipeline

- ✓ Implement model comparison
- ✓ Orchestrate all components

17. Intelligent Training Decisions

- ✓ Develop decision-making system
- ✓ Implement SKIP/FINE_TUNE/RESUME/SCRATCH logic

18. Memory Management

- ✓ Implement memory cleanup
- ✓ Include garbage collection

19. Comprehensive Reporting

- ✓ Generate detailed reports

- ✓ Organize directory structure

20. Visual Documentation

- ✓ Create interactive documentation
 - ✓ Include 3D visualizations
-

1. Data Collection and Preparation

- ✓ Download ASL dataset from Kaggle
- ✓ Verify dataset integrity
- ✓ Split dataset into train/val/test
- ✓ Run MediaPipe cropping on dataset
- ✓ Cache cropped images
- ✓ Analyze class distribution
- ✓ Create data augmentation pipeline

2. Model Development

- ✓ Implement EfficientNetV2B3 builder
- ✓ Implement ResNet50 builder
- ✓ Implement InceptionV3 builder
- ✓ Configure model hyperparameters
- ✓ Set up training callbacks
- ✓ Implement ensemble logic
- ✓ Add TTA support
- ✓ Implement temperature scaling

3. Training Execution

- ✓ Train EfficientNetV2B3 model
- ✓ Train ResNet50 model
- ✓ Train InceptionV3 model
- ✓ Monitor training progress
- ✓ Save best model checkpoints
- ✓ Fine-tune models if needed
- ✓ Export final models to .keras

4. Evaluation and Metrics

- ✓ Generate confusion matrices
- ✓ Calculate accuracy metrics
- ✓ Create ROC curves
- ✓ Create PR curves
- ✓ Generate class distribution plots
- ✓ Create model comparison charts
- ✓ Identify confusion pairs
- ✓ Document evaluation results

5. Application Development (app.py)

- ✓ Implement main() entry point
- ✓ Create sidebar controls
- ✓ Implement Image upload tab
- ✓ Implement Video upload tab
- ✓ Implement Live Camera tab
- ✓ Implement Model Comparison tab

- ✓ Add Grad-CAM visualization
- ✓ Add temporal smoothing
- ✓ Implement error handling
- ✓ Optimize for performance

6. Preprocessing Pipeline

- ✓ Implement CLAHE enhancement
- ✓ Implement gray-world correction
- ✓ Implement gamma correction
- ✓ Create detection fallback chain
- ✓ Implement rotation normalization
- ✓ Add template matching fallback
- ✓ Add motion-based proposals

7. Explainability (XAI)

- ✓ Implement Grad-CAM
- ✓ Add percentile normalization
- ✓ Create landmark-based fallback
- ✓ Implement overlay drawing
- ✓ Add bounding box visualization
- ✓ Add landmark visualization

8. Documentation

- ✓ Write README.md
- ✓ Document installation steps
- ✓ Create usage examples
- ✓ Document API functions
- ✓ Add inline code comments
- ✓ Create architecture diagrams
- ✓ Write troubleshooting guide

9. Testing and Quality

- ✓ Test image upload functionality
- ✓ Test video processing
- ✓ Test live camera mode
- ✓ Test model loading
- ✓ Test ensemble predictions
- ✓ Test edge cases
- ✓ Performance testing
- ✓ Cross-browser testing

10. Deployment

- ✓ Prepare requirements.txt
 - ✓ Prepare packages.txt
 - ✓ Test local deployment
 - ✓ Configure Streamlit secrets
 - ✓ Deploy to Streamlit Cloud
 - ✓ Set up model hosting
 - ✓ Monitor deployed app
 - ✓ Gather user feedback
-

Progress Summary

Category	Total Tasks	Completed	In Progress	Not Started
Data Collection	7	7	0	0
Model Development	8	8	0	0
Training Execution	7	7	0	0
Evaluation	8	8	0	0
Application Development	10	10	0	0
Preprocessing	7	7	0	0
Explainability	6	6	0	0
Documentation	7	7	0	0
Testing	8	8	0	0
Deployment	8	8	0	0
Future Enhancements	7	0	0	7
TOTAL	83	76	0	7

Metric	Value
Completion Percentage	91.6%
Tasks Completed	76/83
Remaining Tasks	7 (Future Enhancements)

Performance Optimization

```
# Enable fast mode for better performance
st.session_state['fast_mode'] = True
st.session_state['fast_max_width'] = 640

# Optimize ensemble settings
st.session_state['video_use_ensemble'] = False
st.session_state['live_skip_frames'] = 3

# Memory management
gc.collect()
```

Security and Privacy

Data Handling

Feature	Description
No Persistent Storage	User images/videos not stored
Local Processing	All inference on user device
Secure Connections	WebRTC and model downloads secured
Token Authentication	Hugging Face model access

Best Practices

Practice	Implementation
Timeout Protection	Socket timeout for downloads

Practice	Implementation
Token Security	Secrets stored in Streamlit secrets
File Verification	Model integrity checks
Error Handling	Robust exception handling

Privacy Considerations

Consideration	Status
No Data Collection	User images never leave device
Clear Data Policies	UI indicates local processing
Opt-in Analytics	Only with explicit consent
Compliance	GDPR, CCPA, accessibility standards

Challenges and Solutions

Challenge	Description	Solution
Dataset Selection	Finding diverse, high-quality dataset	Extensive evaluation of 10 datasets
Skin Tone Bias	Many datasets had uniform skin color	Selected dataset with diverse skin tones
Similar Letters	M/N and A/S confusion	Added more training data and data augmentation
Real-time Performance	Maintaining high FPS	Optimized model architecture
Lighting Conditions	Variable lighting affecting detection	Leveraged MediaPipe robust detection
Hand Orientation	Different angles of hand	Used multi-angle training data

References

- MediaPipe Hands Documentation: <https://google.github.io/mediapipe/solutions/hands.html>
- TensorFlow Documentation: <https://www.tensorflow.org/>
- ASL Alphabet Reference: <https://www.nidcd.nih.gov/health/american-sign-language>
- Kaggle Datasets: <https://www.kaggle.com/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Albumentations Documentation: <https://albumentations.ai/docs/>
- EfficientNetV2 Paper: <https://arxiv.org/abs/2104.00298>
- Grad-CAM Paper: <https://arxiv.org/abs/1610.02391>